ACTIVIDAD INDIVIDUAL: CONSTRUCCIÓN Y DOCUMENTACIÓN DE UNA API CON EXPRESS

Instalación • Rutas HTTP • Middleware • Validación • Pruebas • Git

Andrea Melissa Eraso Montero

Yoli Alexandra Muñoz

Analisiz y Desarrollo de Software

(ADSO)

3312932

Servicio Nacional de Aprendizaje SENA

20/octubre/2026

Popayan, Cauca, Colombia

Introducción

En esta actividad construí una API pequeña con Node.js y Express para recibir solicitudes de registro de personas interesadas en los servicios de una organización. La API ofrece dos rutas de consulta (**GET /inicio** y **GET /informacion**) y una ruta para recibir datos (**POST /solicitudes**) que valida nombre, apellido y correo antes de aceptar la solicitud.

El proyecto usa un middleware propio que registra cada petición en la consola, el middleware **express.json()** para leer cuerpos JSON, una validación específica para la ruta POST y manejadores finales para rutas inexistentes (404) y errores (400 y 500). Al final subí el proyecto a un repositorio remoto con su README. En este documento explico qué hice, por qué lo hice y qué habría pasado si hubiera omitido cada paso.

Investigación: middleware en Express:

1. ¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta?

Un middleware es una función que se ubica en el camino entre la petición que envía el cliente y la respuesta que devuelve el servidor. Recibe tres cosas: la petición (**req**), la respuesta (**res**) y una función llamada **next**. Con ellas puede leer o modificar la petición, terminar el ciclo enviando una respuesta, o pasar el control al siguiente eslabón llamando a next(). Se ejecuta durante el ciclo de la petición, después de que esta llega al servidor y antes de que se envíe la respuesta final, en el mismo orden en que fueron registrados en el código.

2. ¿Qué significan los objetos req y res?

req (request) representa la petición del cliente: contiene el método HTTP, la URL, los encabezados, los parámetros y, si se configura bien, el cuerpo en req.body. **res** (response) es la herramienta para construir la respuesta: con ella defino el código de estado (res.status) y envío el contenido (res.json, res.send).

3. ¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta específica y un middleware de manejo de errores?

- **Global:** se registra con app.use() y se ejecuta en todas las peticiones (por ejemplo, mi middleware de registro y express.json()).
- **De ruta específica:** se coloca como argumento de una ruta concreta, antes de su función final. En mi API, validarSolicitud solo se ejecuta en POST /solicitudes.

- **De manejo de errores:** es el único que recibe cuatro parámetros (err, req, res, next). Express lo llama cuando ocurre un error o se ejecuta next(err), y por eso se registra al final.

4. ¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía JSON?

express.json() lee el cuerpo de las peticiones que llegan con formato JSON, lo convierte en un objeto de JavaScript y lo deja disponible en req.body. Si no se configura, Express no interpreta ese cuerpo: req.body queda vacío o indefinido aunque el cliente sí haya enviado datos. En mi API, la validación respondería "faltan campos" aun cuando el cliente envió nombre, apellido y correo. Además, si el JSON llega mal formado, este middleware genera un error que yo capturo y respondo con 400.

5. ¿Por qué importa el orden en que se registran los middleware y las rutas?

Express recorre los middleware y rutas de arriba hacia abajo y se detiene en cuanto uno envía la respuesta. Por eso: (1) express.json() debe estar antes de las rutas que leen req.body; (2) el middleware de registro debe ir primero para ver todas las peticiones; (3) el 404 debe ir después de todas las rutas, porque si estuviera antes respondería "no existe" incluso a rutas válidas; y (4) el manejador de errores va al final para atrapar lo que falle antes.

6. ¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?

La petición queda "colgada": el servidor no envía nada y el cliente (Postman o el navegador) se queda esperando hasta que se cumple el tiempo de espera. Es un error frecuente al olvidar escribir next() después del console.log.

7. ¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?

Técnicamente ambas son funciones con la misma firma, pero su papel es distinto. El middleware prepara, revisa o registra algo y luego pasa el control con next(). La función de la ruta es el destino final: construye y envía la respuesta al cliente, y normalmente no llama a next().

8. ¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?

- Se aceptarían datos vacíos, incompletos o con correos falsos, y se dañaría la calidad de la información.
- El servidor podría fallar de forma inesperada (por ejemplo, intentar usar nombre.trim() cuando nombre no existe) y responder con un error 500 en lugar de un 400 claro.
- Si falta una comprobación de acceso, cualquier persona podría consultar o enviar información que no le corresponde.
- Se podría responder "éxito" cuando la operación realmente falló, y el cliente creería que su solicitud quedó registrada.

9. Incluye un ejemplo cotidiano, explicado sin código, que ayude a entender la función de un middleware.

Un middleware se parece al vigilante de la entrada de una biblioteca. Cuando llega una persona (la petición), el vigilante anota su hora de ingreso (registro), revisa que lleve su carné (validación) y, si todo está bien, le dice "pase" (next) para que llegue al mostrador, que es la ruta que finalmente la atiende. Si no lleva carné, el vigilante la detiene en la puerta y le explica el motivo (respuesta 400). Y si el vigilante ni la deja pasar ni le dice nada, la persona se queda esperando en la puerta indefinidamente, que es lo que ocurre cuando un middleware olvida llamar a next().

Desarrollo de el reto:

1. Preparación del proyecto

Se crea una carpeta llamada API_Inventario y se abre desde Visual Studio Code. En la terminal se comprueba que Node.js y npm estén instalados:

- node -v
- npm -v

```
C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>node -v
v25.8.1

C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>npm -v
11.17.0
```

Luego se inicializa el proyecto y se instala Express:

- npm init -y
- npm install express

```
C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>npm init -y
Wrote to C:\Users\andre\OneDrive\PROYECTOS\API_Inventario\package.json:

{
  "name": "api_inventario",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>npm install express

added 65 packages, and audited 66 packages in 3s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>
```

Estos comandos preparan el proyecto de Node.js y agregan Express como dependencia.

Instalaremos nodemon

- npm install --save-dev nodemon

```
C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>npm install --save-dev nodemon

added 26 packages, and audited 92 packages in 6s

33 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

C:\Users\andre\OneDrive\PROYECTOS\API_Inventario>
```

también vamos a configurar los comandos en el Package.json para que nuestro servidor funcione correctamente

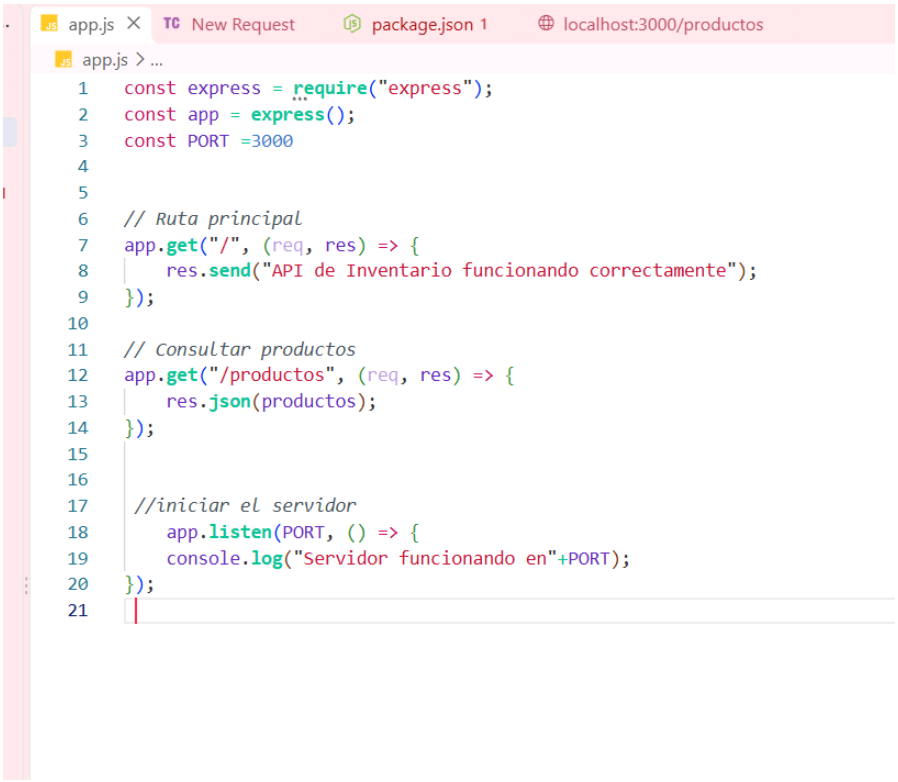


2. Configuración del servidor y rutas

En la carpeta del proyecto se crea el archivo app.js. Allí se importa Express, se crea la aplicación y se define el puerto. Las rutas GET permiten consultar información. Por ejemplo, GET /inicio puede responder con un mensaje de bienvenida



y GET /productos puede mostrar una lista de productos.



3. Ruta POST y validación de datos

Se implementa una ruta POST /productos para recibir los datos de un producto, como su nombre, precio y cantidad. Antes de registrarlo, se validan los datos para comprobar que sean correctos. Si la información no es válida, la API responde con un mensaje de error y el código HTTP 400. Si los datos son correctos, el producto se guarda en la lista y la API confirma su registro mediante el código HTTP 201.

Primero para que funcione debemos agregar esta ruta



Ahora si hacemos el post

```
23  // Registrar un producto
24  app.post("/productos", (req, res) => {
25
26      const { nombre, precio, cantidad } = req.body;
27
28      // Validar los datos
29      if (!nombre || precio <= 0 || cantidad < 0) {
30          return res.status(400).json({
31              mensaje: "Los datos del producto no son válidos"
32          });
33      }
34
35      // Crear el producto
36      const nuevoProducto = {
37          id: productos.length + 1,
38          nombre: nombre,
39          precio: precio,
40          cantidad: cantidad
41      };
42
43      // Guardar el producto
44      productos.push(nuevoProducto);
45
46      // Responder al usuario
47      res.status(201).json({
48          mensaje: "Producto registrado correctamente",
49          producto: nuevoProducto
50      });
51
52  });
```

Crea una ruta POST llamada /solicitudes, (req, res) son los parámetros que representan la petición y la respuesta.

const { nombre, precio, cantidad } = req.body, esta es parte se llama desestructuración de objetos.

Con el if, lo que buscamos es la validación de los datos de la ruta post

4. Middleware

Un middleware es una función que participa en el procesamiento de una solicitud antes de que llegue a la ruta o continúe su recorrido. En este proyecto se puede usar un middleware propio para registrar en la terminal el método HTTP y la ruta

solicitada. La función next() permite continuar con el siguiente paso del procesamiento.

Para lograr esto vamos a usar app.use:



Este codigo permite registrar un middleware que se ejecuta cuando llegan solicitudes a la aplicación.

req: contiene la información de la solicitud que recibe el servidor.

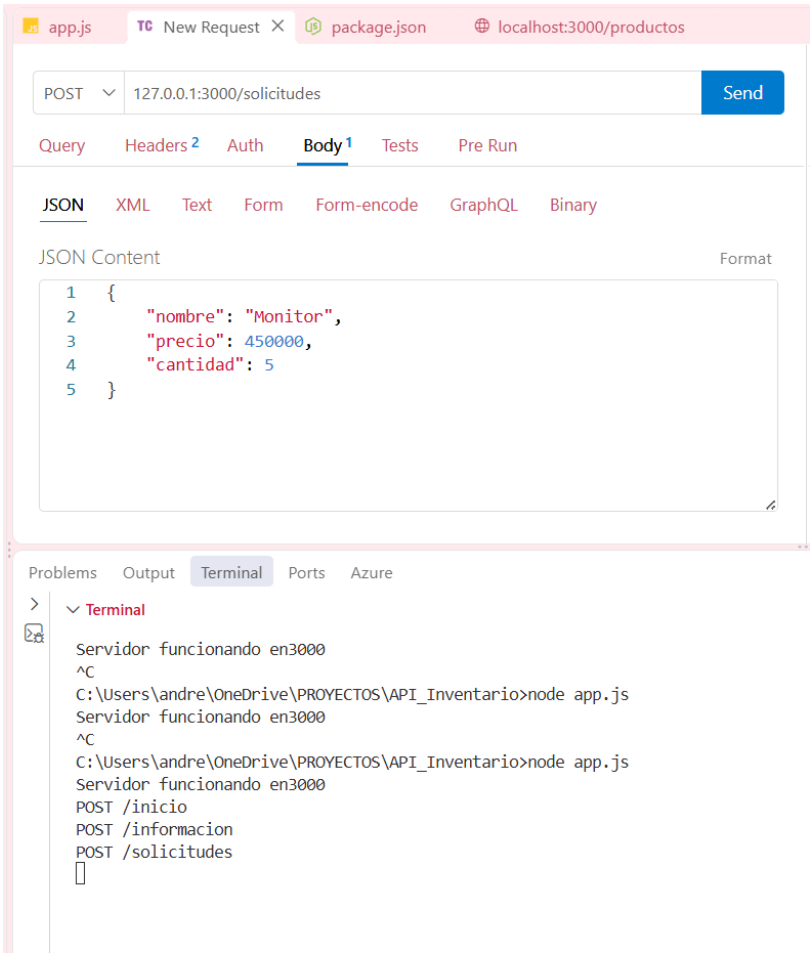
res: permite enviar una respuesta al cliente, aunque en este middleware no lo utilizamos directamente.

next(): indica a Express que continúe con el siguiente middleware o con la ruta correspondiente, si no lo llamamos ni enviamos una respuesta, la solicitud puede quedarse esperando.

console.log(req.method, req.path): muestra en la terminal el método HTTP y la ruta solicitada.

se ubica antes de las rutas Porque así el middleware puede registrar las solicitudes que llegan a las diferentes rutas de la API y no solamente las de una ruta específica.

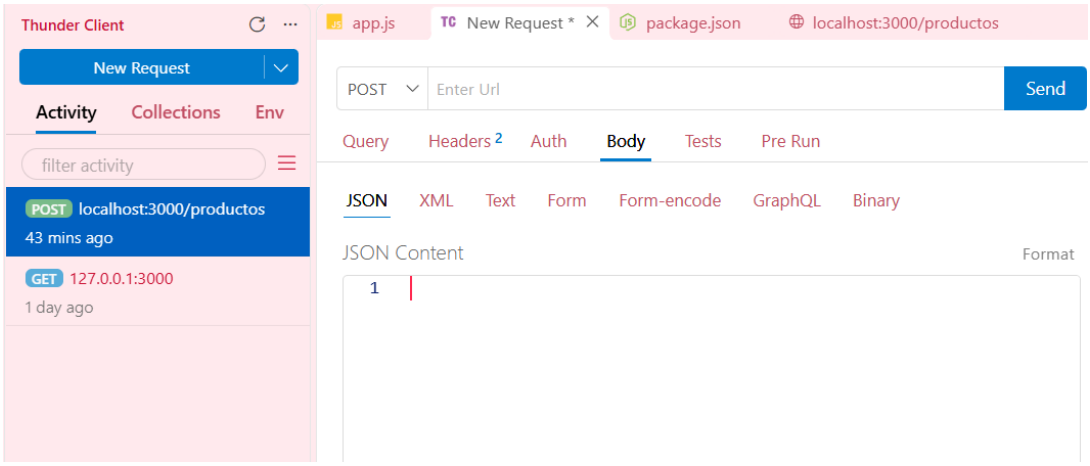
Además, permite observar las peticiones que se realizan durante las pruebas y facilita el seguimiento del funcionamiento de la aplicación.



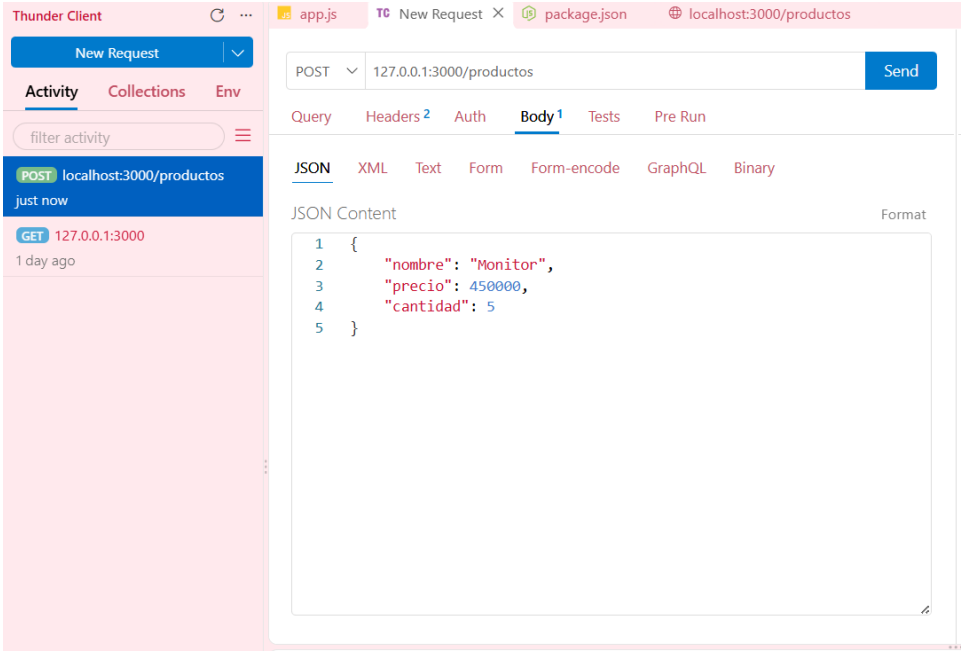
5. Pruebas de la API

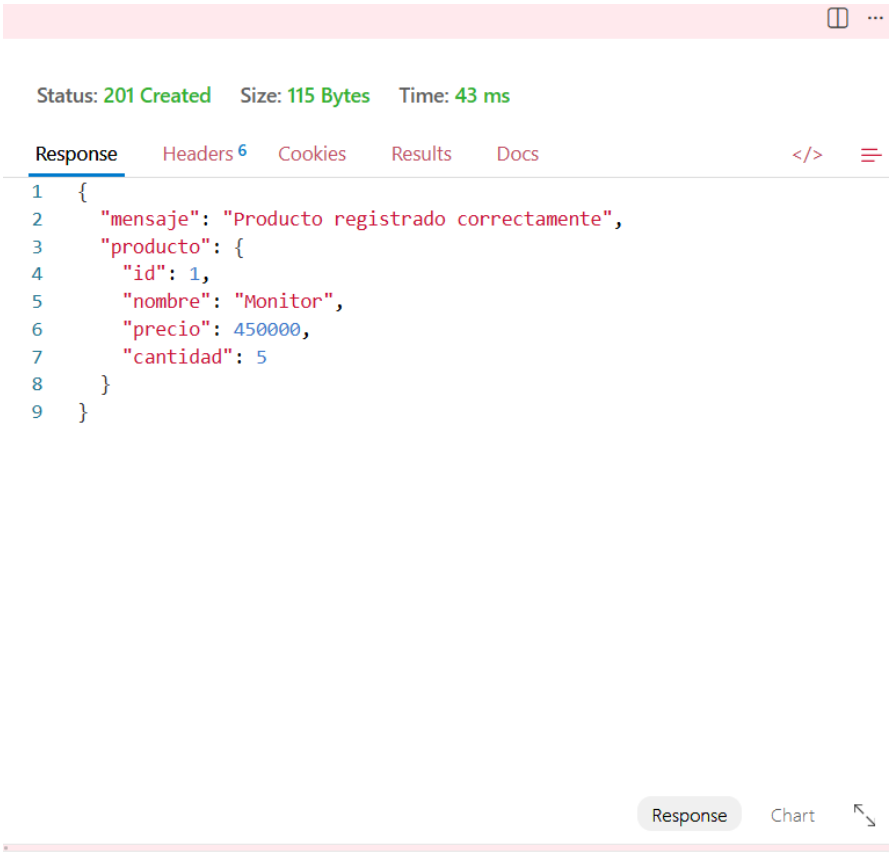
Las rutas se prueban con Thunder Client. Se revisa que las consultas GET respondan y que la ruta POST acepte datos válidos. También se envían datos incompletos o incorrectos para comprobar que la API los rechace de forma clara.

Vamos a seleccionar el método POST, entramos en la pestaña Body. Y seleccionamos el tipo JSON:



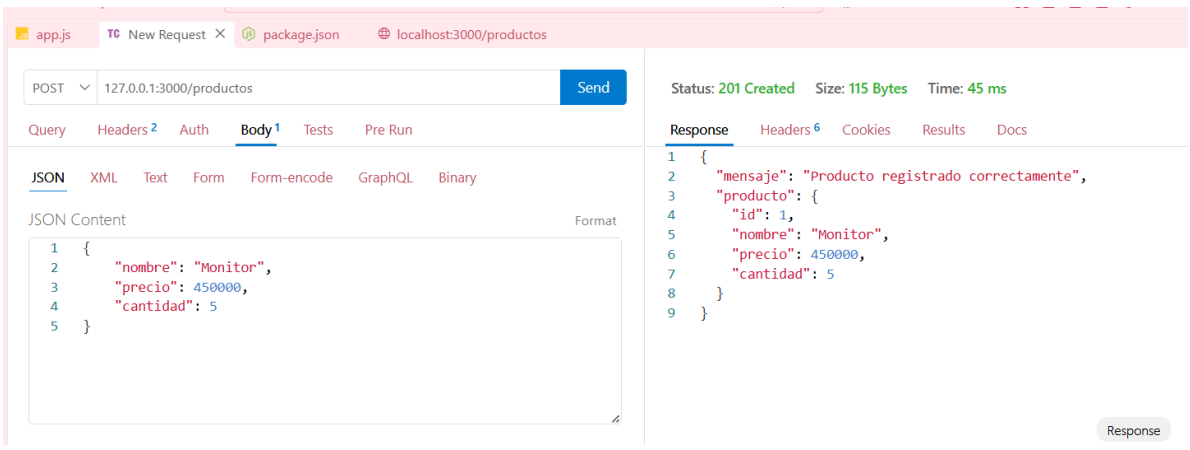
Colocaremos nuestra ruta y la consultaremos algunos datos que le vamos a enviar a uestra appi;





6. Códigos de estado HTTP

Durante las pruebas se pueden observar diferentes códigos de estado. 200 OK indica que una consulta se atendió correctamente; 201 Created se utiliza cuando se confirma la creación de un recurso;



400 Bad Request señala que los datos enviados no son válidos; 404 Not Found indica que la ruta solicitada no existe; y 500 Internal Server Error representa un error inesperado del servidor. Se deben documentar únicamente los códigos que efectivamente se implementaron y probaron.

Conclusión

La construcción de esta API permitió practicar cómo se organiza un servidor con Express y cómo se atienden solicitudes mediante rutas. También se comprendió la utilidad de validar los datos antes de procesarlos y de utilizar middleware para registrar información de las peticiones. Las pruebas ayudaron a comprobar las respuestas de la aplicación tanto cuando los datos son correctos como cuando se presentan errores. Este ejercicio fortalece la comprensión del funcionamiento básico de una API y de la importancia de documentar su desarrollo.

Fuentes de consulta

1. Express. (s. f.). *Using middleware*. https://expressjs.com/en/guide/using-middleware.html
2. Express. (s. f.). *Writing middleware for use in Express apps*. https://expressjs.com/en/guide/writing-middleware.html
3. Express. (s. f.). *Error handling*. https://expressjs.com/en/guide/error-handling.html
4. Express. (s. f.). *Express API reference* (express.json). https://expressjs.com/en/api.html
5. MDN Web Docs. (s. f.). *Códigos de estado de respuesta HTTP*. https://developer.mozilla.org/es/docs/Web/HTTP/Status
6. Node.js. (s. f.). *Documentación de Node.js*. https://nodejs.org/es/docs
7. Nodemon. (s. f.). *Documentación de Nodemon*. https://nodemon.io

8. Git. (s. f.). *Documentación de Git*. https://git-scm.com/doc

- **Fuentes 1, 2 y 3:** preguntas de middleware (qué es, global/de ruta/de errores, next(), orden).
- **Fuente 4:** qué hace express.json() y req/res.
- **Fuente 5:** los códigos 200, 201, 400, 404 y 500.
- **Fuentes 6, 7 y 8:** instalación, Nodemon y Git.